

デバッグ技法

デバッグ(debug)の基本

- バグ(bug)を取り除くこと
- **確認の原則**
 - どの部分がバグなのかなのかを発見する必要がある
 - ➡ 「実際に正しい」動作をしているか順番に確認するしかない
- 効率的な作業の指針
 - デバッガ(debugger)を使う
 - トップダウンに調べる
 - 小さく始める
 - 二分探索を用いる

gdb

- 代表的なデバッガプログラム
 - 実行中のプログラムの状態を制御・確認するツール
 - 任意の場所での一時停止
 - 変数の値の確認, 値の変化の検出
- 使うための準備
 - デバッグ対象のプログラムをコンパイルするときに,
(デバッグ情報を埋め込むために)
gccに **-g** オプションをつける
- 起動方法
 - `$ gdb デバッグ対象の実行プログラム`

gdbの操作方法

- 実行と中断
 - (r)un : デバッグ対象のプログラムの実行
(省略形はr, 以下同じ)
 - Ctrl+Cキー : プログラムの一時停止 (中断)
- ブレークポイント
 - (b)reak *関数名など* : その場所に到達すると自動的に中断
- ステップ実行
 - (n)ext : 中断場所から1行分だけ進める
 - (s)tep : 1行進めるが, 関数呼び出しがあれば中に入る

gdbの操作方法

- 実行再開
 - (c)ontinue : 中断場所から続きを実行する
 - (fin)ish : 関数の呼び出し元に戻るまで進める
- 変数の調査など
 - (p)rint 式(変数名など) : 式の演算結果, 変数の値などを表示
 - set 変数名=値 : 変数の値を変更する
 - (wa)tch 変数名 : 変数の内容(値)が変わると自動的に中断
 - (l)ist : 中断場所周辺のソースを表示

gdbの操作方法

- 関数呼び出し履歴
 - (back)trace : 関数呼び出し履歴が表示される
 - (f)rame フレーム番号 : 履歴内の関数に(確認のために)移動

coreを伴うデバッグ

- coreとは
 - セグメントエラー(Segmentation fault)で異常終了したときの、プログラムの実行状態を保存したファイル
 - gdbに渡すことで、エラー発生時の状態が確認できる
- 使い方
 - `$ gdb エラーを起こしたプログラム core`
 エラーを起こした場所で中断したような状態になっている

エディタとの連携

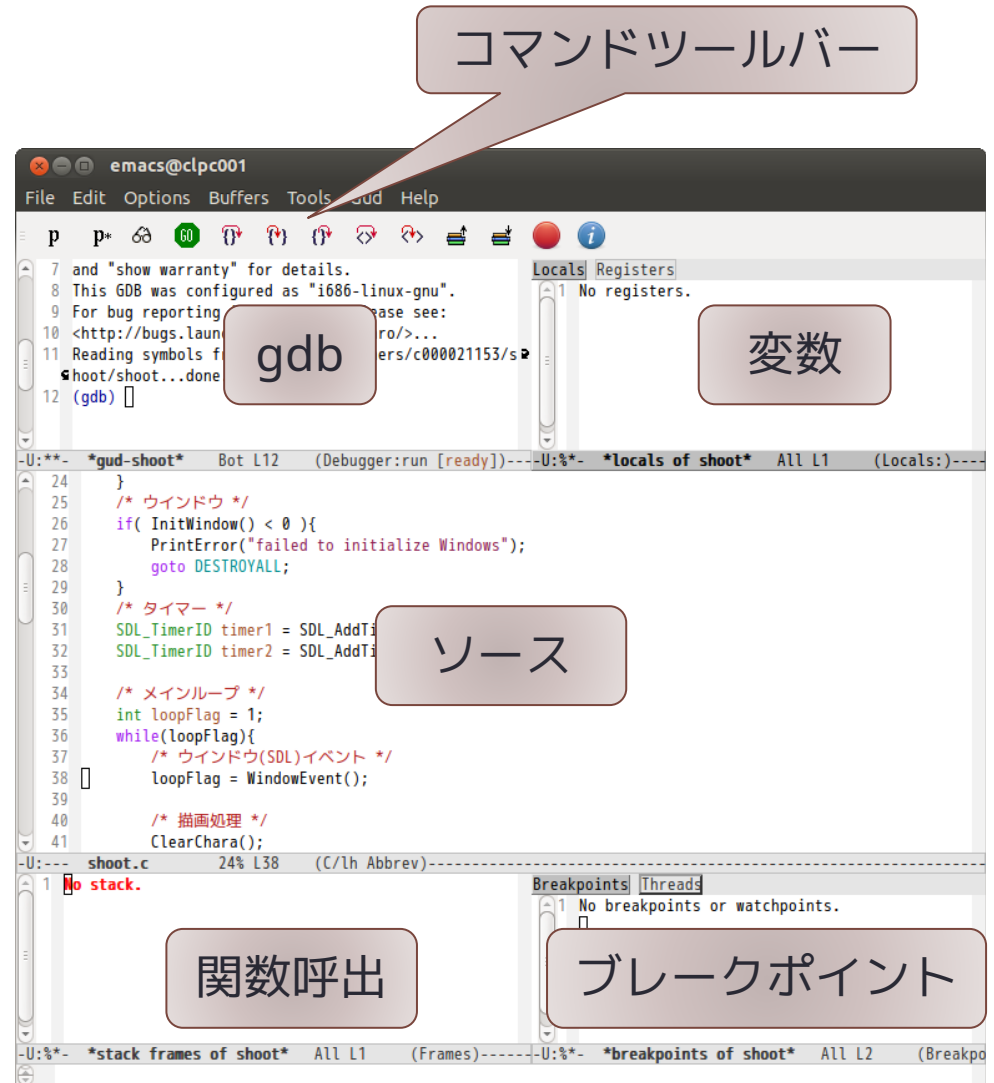
- デバッグはGUI的に使えた方が便利
 - 変数の値などを俯瞰的に確認できる
 - マウスクリックでの実行操作, 変数の確認ができる
- しかしgdbはコマンドラインツール



- エディタ（などGUIのツール）と連携させる
 - emacs
 - Visual Studio Code
 - etc.

emacsとの連携

- emacsを起動
(-nw じゃない方がいい)
- **M-x gdb** とタイプ



Visual Studio Codeとの連携

- Visual Studio Code を起動
- デバッグボタンから開始
 - 事前に設定,
拡張機能の
インストールが必要

コマンドツールバー

デバッグボタン

変数

ソース

関数呼出

コンソール

